



Laporan Praktikum Algoritma & Pemrograman
Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251235
Nama Lengkap	NATALIA GRECIA PUTRI
Minggu ke / Materi	05 / Modular Programming

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

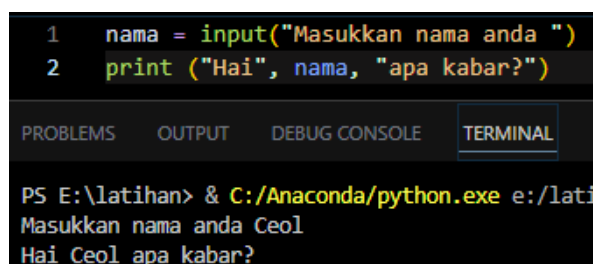
Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

Fungsi, Argument dan Parameter

Fungsi adalah sekumpulan instruksi yang disatukan dalam satu blok guna menyelesaikan pekerjaan tertentu. Python menye diakan dua kategori fungsi:

1. Fungsi bawaan (built-in-function) yang sudah tersedia seperti input() dan print().
2. Fungsi yang dibuat sendiri oleh programmer sesuai kebutuhan.

Contoh Penggunaan Fungsi Bawaan



```
1 nama = input("Masukkan nama anda ")
2 print ("Hai", nama, "apa kabar?")
```

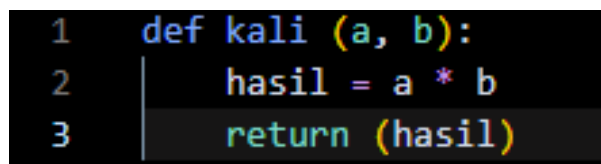
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS E:\latihan> & C:/Anaconda/python.exe e:/lati
Masukkan nama anda Ceol
Hai Ceol apa kabar?

Gambar 5.1: Fungsi bawaan

Contoh penggunaan Fungsi Sendiri

Programmer bisa membuat fungsi menggunakan kata kunci def, diikuti nama fungsi dan parameternya. Sebagai contoh, fungsi perkalian dua bilangan.



```
1 def kali (a, b):
2     hasil = a * b
3     return (hasil)
```

Gambar 5.2: Fungsi Sendiri

Penjelasan:

- Def digunakan untuk mendeklarasikan fungsi.
- Kali (a, b) adalah nama fungsi dengan parameter a dan b.

- Hasil = $a * b$ menghitung perkalian dua angka.
- Return hasil mengembalikan hasil perkalian.

Contoh Penggunaan Fungsi Tambah

```

1  def kali(v, w):
2      hasil= v * w
3      return (hasil)
4
5  c = kali (13, 5)
6  print(c)

```

PROBLEMS OUTPUT **TERMINAL** ...

65

Gambar 5.3: Fungsi Tambah

Hasil dari kode di atas adalah 65, karena fungsi kali(13, 5) mengembalikan nilai $13 * 5$.

Return Value

Fungsi Python terbagi menjadi dua tipe berdasarkan ada tidaknya nilai kembalian.

1. Fungsi yang Tidak Mengembalikan Nilai (Void Function)

Fungsi yang tidak tidak menyertakan return.

Contoh fungsi print_twice() yang hanya mencetak pesan tanpa mengembalikan nilai:

```

1  def print_twice(message):
2      print(message)
3      print(message)
4
5  print_twice("One dream")

```

Gambar 5.4: Fungsi yang Tidak Mengembalikan Nilai (Void Function)

Penjelasan kode:

- Fungsi print_twice() memiliki satu parameter message.
- Fungsi akan mencetak message sebanyak dua kali.
- Karena tidak menggunakan return, fungsi ini tidak mengembalikan nilai apapun.

Apabila hasil fungsi semacam ini dicetak, yang akan ditampilkan adalah None.

```
7 print(print_twice("One dream"))
```

PROBLEMS OUTPUT **TERMINAL** ...

```
PS E:\latihan> & C:/Anaconda/python.exe e:
One dream
One dream
One dream
One dream
None
```

Gambar 5.5: Memanggil print() terhadap hasil print_twice()

Nilai None muncul karena print_twice() tidak mengembalikan nilai apa pun.

2. Fungsi yang Mengembalikan Nilai

Fungsi yang menghasilkan nilai lewat return, misalnya fungsi penghitungan luas dan keliling persegi panjang yang masing-masing mengembalikan hasil perhitungannya ke bagian program yang memanggilnya.

```
1 def hitung_luas_persegi_panjang(panjang, lebar):
2     luas = panjang * lebar
3     return luas
4
5 def hitung_keliling_persegi_panjang(panjang, lebar):
6     keliling = 2 * (panjang + lebar)
7     return keliling
8
9 panjang = 13
10 lebar = 7
11
12 luas = hitung_luas_persegi_panjang(panjang, lebar)
13 keliling = hitung_keliling_persegi_panjang(panjang, lebar)
14
15 print(f"Luas: {luas}")
16 print(f"Keliling: {keliling}")
```

PROBLEMS OUTPUT **TERMINAL** ... Python: fg + -

```
Luas: 91
Keliling: 40
```

Gambar 5.6: Fungsi yang Mengembalikan Nilai

Penjelasan kode:

1. Fungsi `hitung_luas_persegi_panjang(panjang, lebar)`
 - Menerima dua parameter: panjang dan lebar.
 - Mengalikan panjang dengan lebar untuk menghitung luas.
 - Mengembalikan hasil perhitungan luas.
2. Fungsi `hitung_keliling_persegi_panjang(panjang, lebar)`
 - Menerima dua parameter: panjang dan lebar.

- Menggunakan rumus keliling: $2 * (\text{panjang} + \text{lebar})$.
- Mengembalikan hasil perhitungan keliling.

3. Pemanggilan Fungsi

- Nilai panjang = 10 dan lebar = 5 didefinisikan.
- Fungsi `hitung_luas_persegi_panjang()` dipanggil dengan nilai tersebut, dan hasilnya disimpan dalam variabel `luas`.
- Fungsi `hitung_keliling_persegi_panjang()` dipanggil dan hasilnya disimpan dalam variabel `keliling`.

4. Output

- Hasil luas dan keliling ditampilkan menggunakan `print()`.
- Luasnya adalah 50 dan kelilingnya 30.

Optional Argument dan Named Argument

Fungsi dalam Python dapat memiliki parameter operasi yang memiliki nilai bawaan (default). Jika nilai parameter tidak diberikan saat pemanggilan fungsi, maka nilai default akan digunakan. Untuk mendefinisikan parameter opsional, kita harus menentukan nilai bawaan terlebih dahulu.

Selain itu, dalam Python, kita juga bisa menggunakan named arguments saat memanggil fungsi. Artinya, kita dapat menyebutkan nama parameter saat memberikan nilai, sehingga urutan parameter tidak harus sesuai dengan definisi fungsi.

Contoh Named Arguments

```

1 def tampilkan_data(nama, umur, kota):
2     print("Nama: ", nama)
3     print("Umur: ", umur)
4     print("Kota: ", kota)
5
6 #sesuai urutan
7 tampilkan_data("Carmen", 25, "Indonesia")
8
9 #tidak sesuai urutan
10 tampilkan_data(kota="Busan", nama="Daniel", umur=28)

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
Nama: Carmen Umur: 25 Kota: Indonesia Nama: Daniel Umur: 28 Kota: Busan				

Gambar 5.7: Contoh Named Arguments

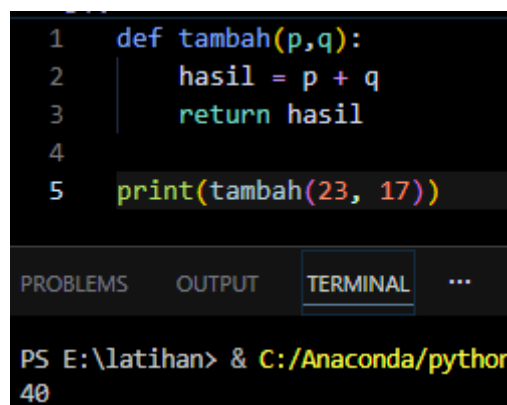
Pada contoh ini, dapat dilihat bahwa meskipun urutan parameter berbeda, hasilnya akan tetap sama.

Anonymous Function (Lambda)

Anonymous function adalah fungsi yang tidak memiliki nama. Dalam Python, anonymous function dikenal sebagai lambda function. Fungsi ini tidak sekompleks fungsi biasa, karena hanya digunakan untuk operasi sederhana dan memiliki keterbatasan dalam hal penulisan beberapa pernyataan.

Berbeda dengan bahasa pemrograman seperti Haskell, Lisp, dan Erlang yang lebih berbasis fungsi, di Python, fungsi lambda lebih sebagai fitur tambahan, bukan fitur utama. Untuk membuat fungsi anonim di Python, kita menggunakan kata kunci lambda.

Contoh 1 fungsi biasa menjumlahkan dua angka:



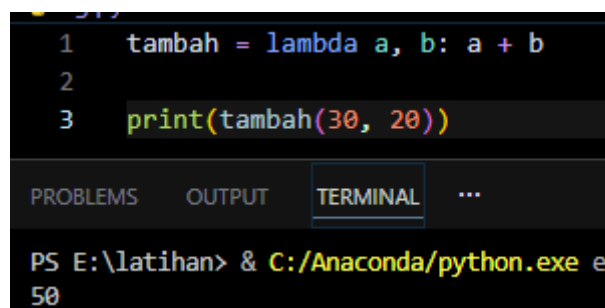
```
1 def tambah(p,q):
2     hasil = p + q
3     return hasil
4
5 print(tambah(23, 17))
```

PROBLEMS OUTPUT TERMINAL ...

PS E:\latihan> & C:/Anaconda/python.exe e
40

Gambar 5.8: ngsi biasa menjumlahkan dua angka

Fungsi ini dapat ditulisk ulang menggunakan lambda function:



```
1 tambah = lambda a, b: a + b
2
3 print(tambah(30, 20))
```

PROBLEMS OUTPUT TERMINAL ...

PS E:\latihan> & C:/Anaconda/python.exe e
50

Gambar 5.9: Penjumlahan dalam bentuk Lambda Function

Struktur lambda Function

Sebuah lambda function dalam Python terdiri dari tiga bagian utama:

1. Keyword lambda: Menandakan bahwa ini adalah lambda function.
2. Bound variable (parameter): Daftar parameter yang diberikan ke fungsi.
3. Body (ekspresi): Pernyataan tunggal yang akan dieksekusi dan mengembalikan nilai.

Contoh 2: Menggunakan Lambda dalam Operasi Lain

Lambda kerap digunakan dalam fungsi higher-order, seperti map(), filter(), dan sorted().

1. Menggunakan lambda dalam map()

```
1 angka = [11,12, 13, 14, 15]
2 kuadrat = list(map(lambda x : x**2, angka))
3 print(kuadrat)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

[121, 144, 169, 196, 225]

Gambar 5.10: Lambda dalam map()

2. Menggunakan lambda dalam filter()

```
1 angka = [10, 15, 30, 35, 50]
2 hasil = list(filter(lambda x: x > 15, angka))
3 print(hasil)
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

[30, 35, 50]

Gambar 5.11: Lambda dalam filter()

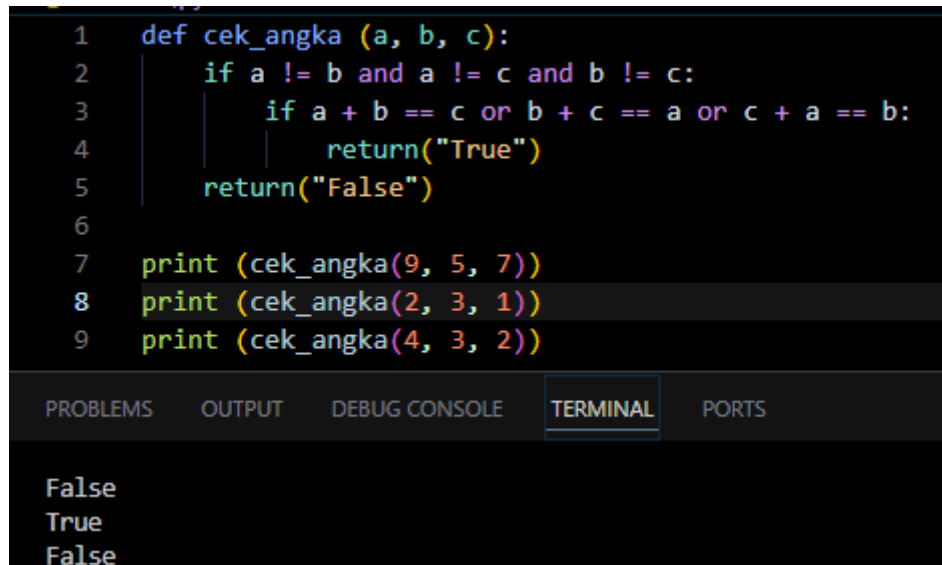
- Lambda function berguna untuk membuat fungsi sederhana dalam satu baris kode.
- Hanya bisa berisi satu ekspresi, tidak bisa memiliki beberapa pernyataan seperti fungsi biasa.
- Sering digunakan dalam fungsi bawaan Python seperti map(), filter(), dan sorted() untuk operasi singkat.

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

https://github.com/grec12/71251235_nataliagrecia.git



```
1 def cek_angka (a, b, c):
2     if a != b and a != c and b != c:
3         if a + b == c or b + c == a or c + a == b:
4             return("True")
5     return("False")
6
7 print (cek_angka(9, 5, 7))
8 print (cek_angka(2, 3, 1))
9 print (cek_angka(4, 3, 2))
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

False
True
False

Gambar 5.12: Fungsi cek angka

- Baris pertama mendefinisikan fungsi `cek_angka` dengan tiga parameter `a`, `b`, dan `c` menggunakan kata kunci `def`. fungsi ini belum langsung dijalankan saat didefinisikan, melainkan baru akan dieksekusi saat dipanggil.
- Baris kedua, kondisi ini menggunakan `if` untuk mengecek apakah ketiga bilangan saling berbeda satu sama lain menggunakan operator `!=` (tidak sama dengan) yang digabungkan dengan operator logika `and`.
- Baris ketiga, salah satu dari tigakemungkinan penjumlahan dua angka harus sama dengan angka ketiga: `a + b == c`, atau `b + c == a`, atau `c + a == b`. kondisi ini dicek dengan operator `or`.
- Jika kedua kondisi terpenuhi, fungsi mengembalikan string `"True"` (baris keempat). Jika tidak, fungsi mengembalikan `"False"` (baris kelima).
- Hasil pemanggilan: `cek_angka(9, 5, 7)` menghasilkan `False`, `cek_angka(2, 3, 1)` menghasilkan `True` karena `2 + 1 = 3`, `cek_angka(4, 3, 2)` menghasilkan `False`.

SOAL 2

```
1 def cek_digit_belakang (a, b, c):
2     digit_a = a % 10
3     digit_b = b % 10
4     digit_c = c % 10
5     return (digit_a == digit_b or digit_b == digit_c or digit_c == digit_a)
6
7 a = int(input("Masukkan angka:"))
8 b = int(input("Masukkan angka:"))
9 c = int(input("Masukkan angka:"))
10 hasil = cek_digit_belakang (a, b, c)
11 print (hasil)
```

Gambar 5.13: Fungsi cek digit belakang

- Baris pertama mendefinisikan fungsi cek_digit_belakang dengan tiga parameter a, b, dan c menggunakan kata kunci def. Fungsi ini belum langsung dijalankan saat didefinisikan, melainkan baru akan dieksekusi saat dipanggil.
- Baris kedua menghitung digit terakhir dari variabel a menggunakan operator modulo (%). Operator modulo menghasilkan sisa hasil bagi, sehingga a % 10 akan menghasilkan digit satuan dari a.
- Baris ketiga dan keempat melakukan hal yang sama untuk variabel b dan c. Operasi b % 10 menghasilkan digit satuan dari b dan disimpan ke variabel digit_b, sedangkan c % 10 menghasilkan digit satuan dari c dan disimpan ke variabel digit_c. Dengan cara ini, ketiga digit satuan dari a, b, dan c sudah tersimpan masing-masing di variabel digit_a, digit_b, dan digit_c.
- Baris kelima, fungsi mengembalikan hasil evaluasi boolean yang mengecek apakah ada setidaknya satu pasangan digit terakhir yang sama, menggunakan operator or.
- Baris ketujuh sampai sembilan, program meminta tiga angka dari pengguna dan mengkonversinya ke integer.

```
Masukkan angka: 30
Masukkan angka: 20
Masukkan angka: 18
True
PS E:\latihan> & C:/Ana
Masukkan angka: 1024
Masukkan angka: 14
Masukkan angka: 94
True
PS E:\latihan> & C:/Ana
Masukkan angka: 53
Masukkan angka: 8900
Masukkan angka: 658
False
```

Gambar 5.14: Output cek digit belakang

SOAL 3

```
1 celcius_to_fahrenheit = lambda C: (9/5) * C + 32
2 celcius_to_reamur = lambda C: 0.8 * C
3
4 print(f"Input C = 100. Output F = {int(celcius_to_fahrenheit(100))}")
5 print(f"Input C = 80. Output R = {int(celcius_to_reamur(80))}")
6 print(f"Input C = 0. Output F = {int(celcius_to_fahrenheit(0))}")
7
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
Input C = 100. Output F = 212
Input C = 80. Output R = 64
Input C = 0. Output F = 32
```

Gambar 5.14: Lambda konversi suhu

- Baris 1: Mendefinisikan lambda function bernama `celcius_to_fahrenheit`. Lambda adalah fungsi anonim (tanpa nama def) yang ditulis dalam satu baris. Rumus konversi adalah $(9/5) * C + 32$, dimana C adalah suhu dalam Celcius.
- Baris 2: mendefinisikan lambda function bernama `celcius_of_reamur` dengan rumus $0.8 * C$.
- Baris 4 – 6: Memanggil kedua fungsi lambda menggunakan f-string untuk mencetak hasil konversi. Fungsi `int()` digunakan untuk membulatkan hasil ke bilangan bulat.
- Hasil: 100 C = 212 F, 80 C = 64 R, 0 C = 32 F.